BD teste 2

2025

Preguntas de 1 a 30 15 valores

1 **Em SQLServer, os índices non-clustered podem existir em qualquer tabela, independentemente de esta ser clustered ou heap : VERDADEIRA**

Explicação: Os índices *non-clustered* são estruturas independentes que contêm apenas as colunas indexadas e um apontador.

2 **Em SQLServer, o fill factor permite reservar espaço livre em páginas de índices para reduzir a ocorrência de page splits. : VERDADEIRA**

Explicação: O *fill factor* define a percentagem de espaço a preencher nas páginas de dados do índice. Deixar espaço libre previene el page split.

3 **A criação de um índice multi-atributo permite acelerar pesquisas mesmo que não se use o primeiro atributo na cláusula WHERE.** : **FALSA**

Explicação: Para que um índice composto (multi-atributo) seja eficaz, a pesquisa deve utilizar obrigatoriamente o primeiro atributo definido no índice izquierdo.

4 **A existência de muitos índices numa tabela nunca afeta negativamente o desempenho de operações de update e delete.** : **FALSA**

Explicação: Sempre que uma linha é inserida, el sgbd hace cambion luego desempeno menor.

5 **Um heap table no SQLServer mantém sempre os dados ordenados fisicamente segundo a ordem da chave primária. : FALSA**

Explicação: Uma tabela do tipo *heap* é uma tabela sem índice *clustered*, o que significa que os dados são armazenados desordenados.

6 **Quando uma batch termina, todas as variáveis locais declaradas nela são automaticamente destruídas.** : **VERDADEIRA**

Explicação: O ciclo de vida das variáveis locais em T-SQL está estritamente limitado à *batch* onde foram criadas. Cuando llega a go explota.

7 **Em SQLServer, um erro de runtime numa batch anula automaticamente todas as instruções já executadas nessa batch.** : **FALSA**

Explicação: Por padrão, um erro de tempo de execução (*runtime*) interrompe a execução das instruções seguintes na *batch*, mas não anula as instruções SQL.

8 **As variáveis do tipo tabela (@Table) não podem ter chaves estrangeiras ou triggers associados.** : **VERDADEIRA**

Explicação: As variáveis de tabela são estruturas simplificadas em memória que possuem várias limitações quando comparadas com tabelas normais .

9 **Cursores permitem percorrer tuplos de uma tabela um a um, o que em geral é mais eficiente que soluções set-based.** : **FALSA**

> **Explicação:** Os cursores processam os dados linha a linha (*row-by-row*), o que consome muitos recursos computacionais.

10 **Triggers INSTEAD OF substituem completamente a execução da instrução DML original.** : **VERDADEIRA**

> **Explicação:** Ao contrário dos *triggers AFTER*, o *trigger INSTEAD OF* é executado **em vez da** ação que o disparou, permitindo desviar ou customizar totalmente o comportamento da operação.

11 **As funções definidas pelo utilizador (UDF) podem conter instruções de controlo de fluxo como TRY...CATCH.** : **FALSA**

> **Explicação:** o uso de blocos TRY...CATCH não é permitido dentro de funções apenas em *Stored Procedures* e *Triggers*.

12 **As tabelas lógicas INSERTED e DELETED podem ser diretamente consultadas dentro de stored procedures chamadas a partir de triggers.** : **FALSA**

> **Explicação:** As tabelas especiais inserted e deleted são locais e exclusivas ao contexto de execução do *trigger*. .

13 **O comando RAISERROR pode ser usado dentro de triggers e stored procedures para forçar mensagens de erro, mas não faz o rollback da transação.** : **VERDADEIRA**

> **Explicação:** O RAISERROR serve para gerar uma mensagem de erro customizada para a aplicação e definir a severidad , el rollback no lo genera.

14 **Em SQLServer, @@ERROR e @@ROWCOUNT reportam respetivamente o código de erro e o número de linhas afetadas.** : **VERDADEIRA**

> **Explicação:** Estas são funções globais .

15 **O estado Partially Committed ocorre imediatamente antes do commit ser garantido fisicamente no disco.** : **VERDADEIRA**

> **Explicação:** Uma transação entra no estado *partially committed* assim que executa a sua última operação na memória. Antes de meterlo al disco.

16 **A propriedade Durability do modelo ACID garante que nenhuma falha posterior pode anular as alterações de uma transação com commit efetuado. VERDADEIRA**

> **Explicação:** A durabilidade garante que, uma vez confirmado o *commit*, os efeitos da transação tornam-se permanentes no sistema.

17 **Num escalonamento recuperável, uma transação T2 pode fazer commit mesmo que tenha lido dados atualizados por T1, ainda não terminado.** : **FALSA**

> **Explicação:** Para ser considerado recuperável , se T2 lê dados modificados por T1, o *commit* de T2 deve obrigatoriamente ser adiado e executado apenas **após** o *commit* de T1.

18 **O método otimista de controlo de concorrência nunca provoca aborts de transações, apenas atrasos de execução.** : **FALSA**

> **Explicação:** O método otimista não usa bloqueios e deixa as transações avancen libre pero si encuentra un error la retrasa.

19 **Em SQLServer, o transaction log pode ser eliminado periodicamente após a execução de commits.** : **FALSA**

> **Explicação:** O *transaction log* nunca é simplesmente eliminado após um *commit*, pois ele é vital para operações de *Redo/Undo*.

20 **A utilização de Savepoints permite efetuar rollbacks parciais dentro de uma transação.** : **VERDADEIRA**

> **Explicação:** Os *savepoints* funcionam como marcos ou pontos de controlo intermédios. Permiten hacer rollbacks.

21 **O checkpoint armazena no transaction log todas as queries executadas após o último commit, servindo como histórico completo de recuperação.** : **FALSA**

> **Explicação:** O *checkpoint* é um mecanismo que força a escrita de todas as páginas de dados "sujas" (em memória) para o disco rígido.

22 **Em SQLServer, um user pode pertencer simultaneamente a múltiplas database roles.** : **VERDADEIRA**

> **Explicação:** Os utilizadores de uma base de dados no SQL Server podem ser associados a vários papéis (*roles*) em simultâneo.

23 **O comando DENY tem sempre prioridade sobre GRANT na resolução de permissões.** **VERDADEIRA**

> **Explicação:** No modelo de segurança do SQL Server, uma proibição explícita (DENY) sobrepõe-se sempre a qualquer permissão concedida (GRANT).

24 **A recuperação de falha de disco utiliza os backups e aplica as before-images do transaction log.** : **FALSA**

> **Explicação:** Para recuperar de uma falha de disco, restaura-se o último backup e aplicam-se as **after-images** .

25 **SQL Injection não permite executar instruções DDL como, por exemplo, DROP TABLE.** : **FALSA**

Explicação: Se a aplicação for vulnerável un atacante puede.

26 **Uma relação com chave simples (não composta) não pode violar a 2FN.** : **VERDADEIRA**

Explicação: A Segunda Forma Normal 2FN proíbe dependências funcionais parciais .

27 **Se uma relação está na 3FN, não pode existir qualquer dependência funcional entre atributos não-chave.** : **VERDADEIRA**

Explicação: A Terceira Forma Normal 3FN exige que a relação esteja na 2FN e que nenhum atributo não-chave dependa de forma transitiva da chave primária. Logo, proíbe diretamente dependências funcionais entre atributos que não pertencem a chaves.

28 **Na decomposição em BCNF nunca há perda de dependências funcionais.** : **FALSA**

Explicação: hay casos donde si .

29 **No processo de normalização de uma base de dados relacional, a redundância está associada a dependências funcionais não desejadas.** : **VERDADEIRA**

Explicação: A redundância ocorre quando guardamos o mesmo facto várias vezes.

30 **Em SQL, as Views podem ser parametrizadas.** : **FALSA**

Explicação: as **Views não aceitam parâmetros de entrada**

```
go
-- Define un tipo de tabla personalizado que acepta una lista de números de proyecto (Pno)
CREATE TYPE ProjectListType AS TABLE (
    Pno INT
);

go
-- Stored procedure que transfiere un empleado a un nuevo departamento
-- y lo registra en los proyectos indicados en @ProjectList
CREATE PROCEDURE TransferEmployeeDepartment
    @Ssn CHAR(9),
    @NewDno INT,
    @ProjectList ProjectListType READONLY  -- tabla en memoria, no se puede modificar dentro del proc
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION  -- todo lo que sigue es atómico: o se confirma todo o nada

        -- Verificación 1: el empleado debe existir
        IF NOT EXISTS (SELECT 1 FROM EMPLOYEE WHERE Ssn = @Ssn)
        BEGIN
            RAISERROR ('Error .....', 16, 1)  -- lanza error de severidad 16 (línea X)
            ROLLBACK TRAN
            RETURN
        END

        -- Verificación 2: el nuevo departamento debe existir
        IF NOT EXISTS (SELECT 1 FROM DEPARTMENT WHERE Dnumber = @NewDno)
        BEGIN
            RAISERROR ('Error .....', 16, 1)
            ROLLBACK TRAN
            RETURN
        END
```

```
        -- Verificación 3: el empleado no puede ser manager de otro departamento distinto al destino
        IF EXISTS (
            SELECT 1 FROM DEPARTMENT
            WHERE Mgr_ssn = @Ssn AND Dnumber <> @NewDno
        )
        BEGIN
            RAISERROR ('Error .....', 16, 1)
            ROLLBACK TRAN
            RETURN
        END

        -- Actualiza el departamento del empleado
        UPDATE EMPLOYEE SET Dno = @NewDno WHERE Ssn = @Ssn

        -- Elimina todas las asignaciones previas del empleado en proyectos
        DELETE FROM WORKS_ON WHERE Essn = @Ssn

        -- Cursor para recorrer los proyectos de la lista uno a uno
        DECLARE @Pno INT

        DECLARE proj_cursor CURSOR FOR
            SELECT Pno FROM @ProjectList

        OPEN proj_cursor
        FETCH NEXT FROM proj_cursor INTO @Pno  -- obtiene el primer proyecto
```

```
        -- Recorre cada proyecto de la lista
        WHILE @@FETCH_STATUS = 0
        BEGIN
            -- Verificación: el proyecto debe existir en la tabla PROJECT
            IF NOT EXISTS (SELECT 1 FROM Project WHERE Pnumber = @Pno)
            BEGIN
                RAISERROR ('Error .....', 16, 1);
                ROLLBACK TRAN;
                RETURN
            END

            -- Inserta el empleado en el proyecto con 0 horas iniciales
            INSERT INTO WORKS_ON (Essn, Pno, Hours) VALUES (@Ssn, @Pno, 0)

            FETCH NEXT FROM proj_cursor INTO @Pno  -- avanza al siguiente proyecto
        END

        -- Libera el cursor de memoria
        CLOSE proj_cursor
        DEALLOCATE proj_cursor

        COMMIT TRAN  -- confirma todos los cambios si no hubo errores

    END TRY
    BEGIN CATCH
        -- Si ocurre cualquier error no controlado, lanza mensaje y deshace todo
        RAISERROR ('Error .....', 16, 1);
        ROLLBACK TRAN
    END CATCH
END
go
```

```
------------------------------------------------- test

-- Bloque de prueba de la stored procedure
DECLARE @Ssn VARCHAR(9)
@Ssn = '123456789'

-- Crea una variable del tipo tabla definido arriba
DECLARE @Projects ProjectListType;
INSERT INTO @Projects (Pno) VALUES (10), (20), (30);  -- lista de proyectos destino

-- Ejecuta la transferencia: empleado 123456789 pasa al departamento 5
-- y se le asignan los proyectos 10, 20 y 30
EXEC TransferEmployeeDepartment @Ssn = '123456789', @NewDno = 5, @ProjectList = @Projects;
```

Esta SP garante a integridade dos dados validando a existência de funcionários e departamentos.
Esta SP permite que funcionários trabalhem em vários departamentos.
Esta SP garante que um funcionário não possa gerenciar vários departamentos.
Esta SP impede a exclusão de atribuições de projetos antigas do funcionário.
Esta SP usa um cursor para iterar sobre uma lista de projetos passada como um parâmetro com valor de tabela.
Esta SP ignora projetos inexistentes e continua com os demais.
Esta SP insere 0 horas de trabalho ao reatribuir um funcionário a novos projetos.
Esta SP permite a inserção de números de projetos inválidos.
Esta SP confirma as alterações mesmo se um erro for gerado durante a inserção do projeto.
Esta SP suporta a reatribuição de funcionários a vários departamentos em uma única execução.
Esta SP gera um erro se um projeto na lista não existir na tabela PROJETO.
Este SP atribui funcionários como gerentes do novo departamento.
Este SP verifica se o funcionário já trabalha nos novos projetos antes da inserção.
Este SP registra mensagens de erro detalhadas do sistema usando ERROR_MESSAGE().
Este SP permite a inserção de horas negativas em WORKS_ON.

1. "Esta SP garante a integridade dos dados validando a existência de funcionários e departamentos."

→ VERDADEIRA. El código tiene dos IF NOT EXISTS que verifican que el empleado (EMPLOYEE WHERE Ssn = @Ssn) y el departamento (DEPARTMENT WHERE Dnumber = @NewDno) existan antes de proceder.

2. "Esta SP garante que um funcionário não possa gerenciar vários departamentos."

→ VERDADEIRA. El tercer bloque IF EXISTS comprueba que el empleado no sea Mgr_ssn de otro departamento distinto al destino. Si lo es, lanza error y hace ROLLBACK.

3. "Esta SP usa um cursor para iterar sobre uma lista de projetos passada como um parâmetro com valor de tabela."

→ VERDADEIRA. El cursor proj_cursor hace SELECT Pno FROM @ProjectList, que es exactamente el TVP (ProjectListType) pasado como parámetro.

4. "Esta SP insere 0 horas de trabalho ao reatribuir um funcionário a novos projetos."

→ VERDADEIRA. La línea INSERT INTO WORKS_ON (Essn, Pno, Hours) VALUES (@Ssn, @Pno, 0) inserta siempre Hours = 0.

5. "Esta SP gera um erro se um projeto na lista não existir na tabela PROJETO."

→ VERDADEIRA. Dentro del WHILE, hay un IF NOT EXISTS (SELECT 1 FROM Project WHERE Pnumber = @Pno) que lanza RAISERROR y hace ROLLBACK

Teste 2024

1. **Em termos de SQL DCL, um GRANT sobrepõe-se a um DENY : FALSA**

   Explicação: deny prioridad sobre un grant.

2. **Em SQL Server, podemos definir o nível de isolamento de uma transação de forma a permitir situações de "leitura suja" : VERDADEIRA**

   Explicação: Definir o nível de isolamento , permite que a transação leia dados que foram modificados por outras transações y aun no dio commit.

3. **No controle de concorrência de transações, o mecanismo de locking é um método do tipo otimista : FALSA**

   Explicação: O mecanismo de *locking* (bloqueios) é a base do controlo de concorrência pessimista.

4. **Um Trigger do tipo After deve ser utilizado quando sabemos que a operação (insert/update/delete) tem uma elevada probabilidade de ser "rolled back"; : FALSA**

   Explicação: Os *triggers AFTER* executam depois de a operação já ter sido realizada na tabela.

5. **Num escalonamento concorrente, podemos ter situações de conflito quando temos transações simultâneas de consulta de dados; : FALSA**

   - **Explicação: Operações de leitura pura (read-read) nunca entram em conflito entre si consulara de datos.**

6. **Em SQL Server, um Stored Procedure não pode ser utilizado como fonte de dados de uma consulta (i.e. na cláusula FROM de um SELECT); : VERDADEIRA**

   Explicação: Não podes colocar um EXEC stored_procedure diretamente dentro da cláusula FROM usar UDF.

7. **Em SQL Server, faz sentido ter um fillfactor de 85% para um índice cujo atributo é do tipo inteiro com inserções ordenadas (por exemplo, identity); : FALSA**

   Explicação: Se as inserções são estritamente ordenadas (como um IDENTITY), os novos dados são sempre adicionados ao fim do índice.

8. **Um escalonamento E diz-se recuperável se nenhuma transação Ti em E for concluída (commited) até que todas as outras transações que escrevem elementos lidos por Ti tenham sido concluídas; : VERDADEIRA**

   - **Explicação: Garante que se uma transação depender de dados de outra, ela só faz *commit* depois da transação produtora também fazer, evitando commits de dados "fantasma".**

9. **Em SQL Server, podemos ter distintos DB users (um por cada DB) mapeados para o mesmo login; : VERDADEIRA**

   Explicação: O *Login* serve para a autenticação no servidor global. Depois.

10. **Num Trigger do tipo After Delete, a tabela lógica inserted pode conter tuplos (i.e. não está vazia); : FALSA**

    Explicação: Numa operação de eliminação (DELETE), os dados antigos removidos vão exclusivamente para a tabela lógica deleted.

11. **Em SQL Server, a ocorrência de um erro numa operação dentro de um "bloco try-catch" leva ao rollback de todas as operações anteriores da transação; FALSA**

    Explicação: O bloco TRY…CATCH apenas captura o erro e desvia o fluxo do código para o bloco CATCH.

12. **Numa UDF, não é possível definir uma transacção (BEGIN TRANSACTION …);**

    VERDADEIRA

    Explicação: Elas não podem alterar o estado da base de dados nem gerir transações.

13. **A variável global @@error do SQL Server contém o código do erro da última instrução; VERDADEIRA**

    Explicação: A função/variável de sistema @@ERROR armazena o número do erro gerado pela instrução SQL imediatamente anterior.

14. **Em SQL Server, os Triggers são disparados uma vez por cada tuplo afetado pela operação (insert/update/delete); : FALSA**

    Explicação: *trigger* dispara apenas uma vez por cada comando executado

15. **Uma dependência funcional é uma propriedade do esquema de relação R que pode ser inferido de uma qualquer instância de R; FALSA**

    Explicação: As dependências funcionais são restrições semânticas do mundo real que definem o esquema da base de dados (R)

16. **Uma apresentação de informação de debug bastante detalhada nos formulários gráficos de uma base de dados ajuda a prevenir situações de SQL Injection; FALSA**

    Explicação: Para prevenir SQL Injection, devem usar-se *Parameterized Queries* (consultas parametrizadas). Mostrar tu cartera te la roban

17. **Quando há uma falha numa transação, é feito um rollback com reposição da versão original dos dados alterados utilizando as "after-image" do Transaction Log; : FALSA**

    Explicação: O ROLLBACK (operação de UNDO) utiliza as before-images (valores antigos antes da alteração) para restaurar o estado original. As *after-images* (valores novos) são usadas na operação de REDO (refazer) durante a recuperação .

18. **Em SQL Server, um atributo definido como unique aceita valores null repetidos; FALSA**

    Explicação: SQL Server implementa a restrição UNIQUE permitindo apenas um único valor NULL.

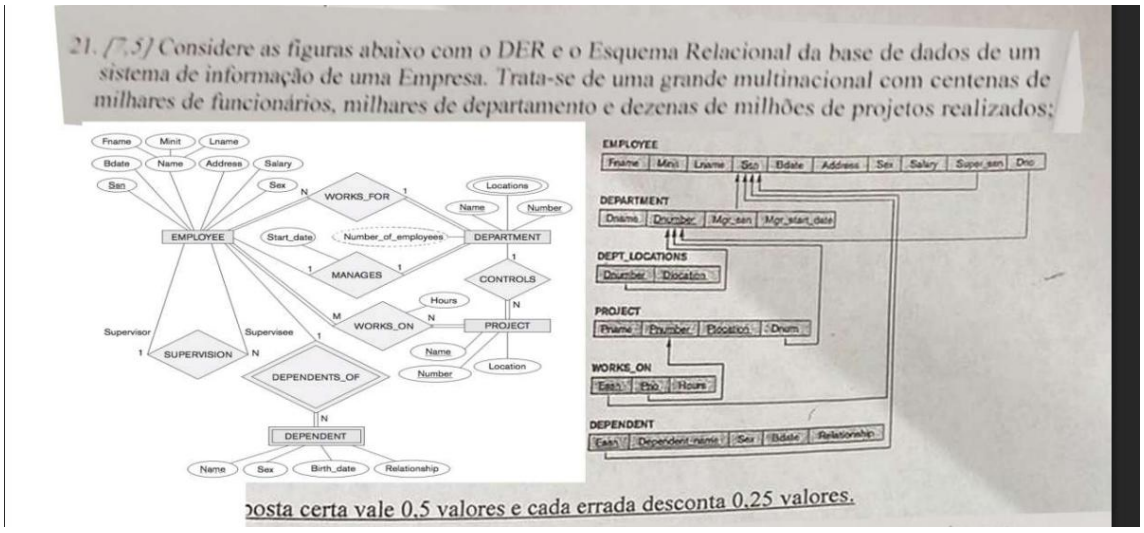
19. **Um cursor permite percorrer sequencialmente os tuplos retornados por uma query definida para o efeito; VERDADEIRA**

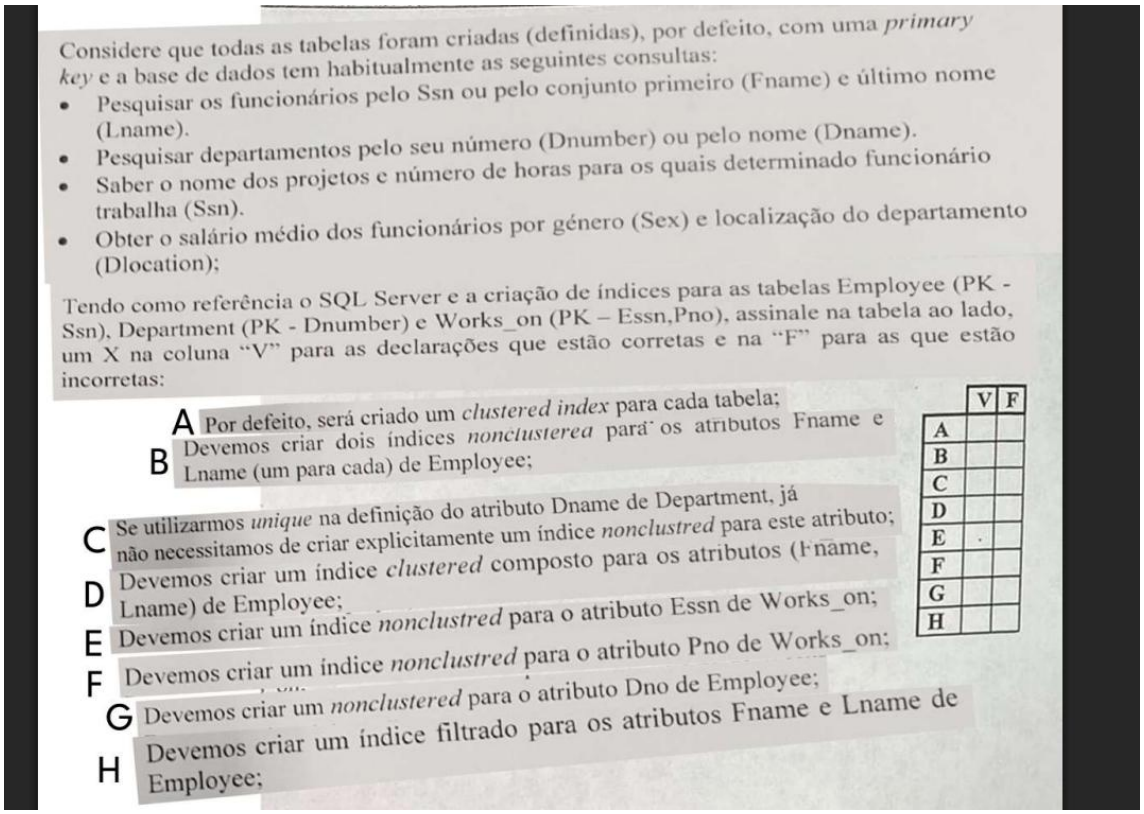
    Explicação: O objetivo principal de um cursor é carregar um conjunto de resultados e permitir que o programador utilize uma estrutura de repetição para processar esses dados linha a linha (row-by-row).

20. **Em SQL Server, as funções EncryptByPassPhrase e DecryptByPassPhrase permitem guardar dados sensíveis de forma cifrada na base de dados, utilizando uma senha que deve ser guardada fora da Base de Dados;**

    VERDADEIRA

    Explicação: Estas funções realizam criptografia simétrica baseada numa chave (passphrase) fornecida em runtime pela aplicação.





Considere que todas as tabelas foram criadas (definidas), por defeito, com uma *primary key* e a base de dados tem habitualmente as seguintes consultas:

- Pesquisar os funcionários pelo Ssn ou pelo conjunto primeiro (Fname) e último nome (Lname).
- Pesquisar departamentos pelo seu número (Dnumber) ou pelo nome (Dname).
- Saber o nome dos projetos e número de horas para os quais determinado funcionário trabalha (Ssn).
- Obter o salário médio dos funcionários por género (Sex) e localização do departamento (Dlocation);

Tendo como referência o SQL Server e a criação de índices para as tabelas Employee (PK - Ssn), Department (PK - Dnumber) e Works_on (PK – Essn,Pno), assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas:

A Por defeito, será criado um *clustered index* para cada tabela;

B Devemos criar dois índices *nonclusterea* para os atributos Fname e Lname (um para cada) de Employee;

C Se utilizarmos *unique* na definição do atributo Dname de Department, já não necessitamos de criar explicitamente um índice *nonclustred* para este atributo;

D Devemos criar um índice *clustered* composto para os atributos (Fname, Lname) de Employee;

E Devemos criar um índice *nonclustred* para o atributo Essn de Works_on;

F Devemos criar um índice *nonclustred* para o atributo Pno de Works_on;

G Devemos criar um *nonclustered* para o atributo Dno de Employee;

H Devemos criar um índice filtrado para os atributos Fname e Lname de Employee;

	V	F
A		
B		
C		
D		
E		
F		
G		
H		

A. Por defeito, será criado um *clustered index* para cada tabela.VERDADEIRO

En SQL Server, al definir una PK se crea automáticamente un *unique clustered index*

B. Devemos criar dois índices *nonclustered* para os atributos Fname e Lname (um para cada) de Employee. FALSO

Se debe crear un solo índice compuesto (Fname, Lname), no dos separados, ya que las búsquedas usan ambos a la vez.

C. Se utilizarmos *unique* na definição do atributo Dname de Department, já não necessitamos de criar explicitamente um índice *nonclustered*.VERDADEIRO

La restricción UNIQUE ya crea implícitamente un índice *nonclustered* en SQL Server.

D. Devemos criar um índice *clustered* composto para os atributos (Fname, Lname) de Employee. FALSO

Employee ya tiene su *clustered index* en la PK (Ssn). Solo puede existir uno por tabla.

E. Devemos criar um índice *nonclustered* para o atributo Essn de Works_on.
FALSO

Essn es el primer atributo de la PK compuesta (Essn, Pno), por lo que las búsquedas por Essn ya están cubiertas por el *clustered index*.

F. Devemos criar um índice *nonclustered* para o atributo Pno de Works_on.
VERDADEIRO

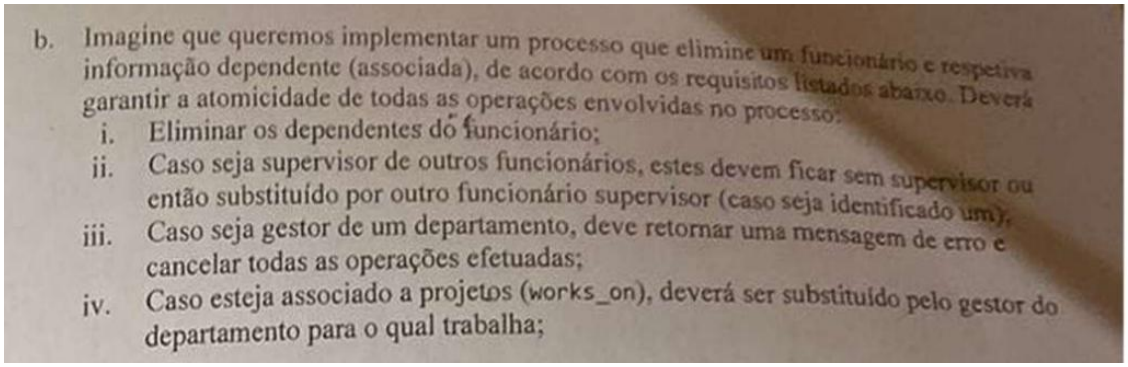
Pno es el segundo atributo de la PK; un índice compuesto solo aplica si la query incluye el primer atributo, por lo que Pno solo necesita su propio índice.

G. Devemos criar um *nonclustered* para o atributo Dno de Employee.
VERDADEIRO

Las consultas de salario medio por Dlocation hacen JOIN entre Employee y Department usando Dno, por lo que indexarlo mejora el rendimiento.

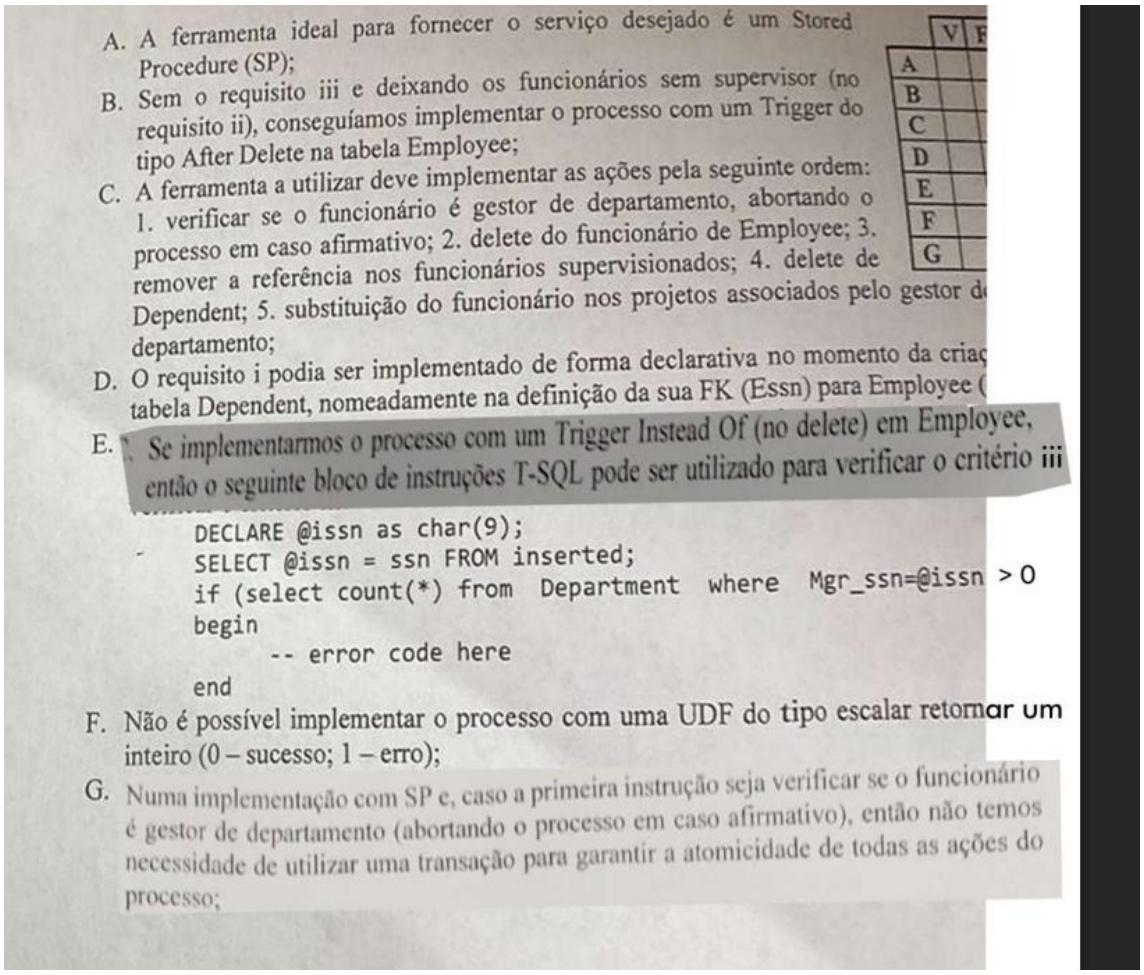
H. Devemos criar um índice filtrado para os atributos Fname e Lname de Employee.
FALSO

Un índice filtrado indexa solo un subconjunto de filas (con cláusula WHERE). Aquí lo correcto es un índice *nonclustered* compuesto normal, sin filtro.



b. Imagine que queremos implementar um processo que elimine um funcionário e respetiva informação dependente (associada), de acordo com os requisitos listados abaixo. Deverá garantir a atomicidade de todas as operações envolvidas no processo:

i. Eliminar os dependentes do funcionário;
ii. Caso seja supervisor de outros funcionários, estes devem ficar sem supervisor ou então substituído por outro funcionário supervisor (caso seja identificado um);
iii. Caso seja gestor de um departamento, deve retornar uma mensagem de erro e cancelar todas as operações efetuadas;
iv. Caso esteja associado a projetos (works_on), deverá ser substituído pelo gestor do departamento para o qual trabalha;



A. A ferramenta ideal para fornecer o serviço desejado é um Stored Procedure (SP);

B. Sem o requisito iii e deixando os funcionários sem supervisor (no requisito ii), conseguíamos implementar o processo com um Trigger do tipo After Delete na tabela Employee;

C. A ferramenta a utilizar deve implementar as ações pela seguinte ordem: 1. verificar se o funcionário é gestor de departamento, abortando o processo em caso afirmativo; 2. delete do funcionário de Employee; 3. remover a referência nos funcionários supervisionados; 4. delete de Dependent; 5. substituição do funcionário nos projetos associados pelo gestor d departamento;

D. O requisito i podia ser implementado de forma declarativa no momento da criaç tabela Dependent, nomeadamente na definição da sua FK (Essn) para Employee (

E. Se implementarmos o processo com um Trigger Instead Of (no delete) em Employee, então o seguinte bloco de instruções T-SQL pode ser utilizado para verificar o critério iii

```
DECLARE @issn as char(9);
SELECT @issn = ssn FROM inserted;
if (select count(*) from Department where Mgr_ssn=@issn > 0
begin
    -- error code here
end
```

F. Não é possível implementar o processo com uma UDF do tipo escalar retornar um inteiro (0 – sucesso; 1 – erro);

G. Numa implementação com SP e, caso a primeira instrução seja verificar se o funcionário é gestor de departamento (abortando o processo em caso afirmativo), então não temos necessidade de utilizar uma transação para garantir a atomicidade de todas as ações do processo;

	V	F
A		
B		
C		
D		
E		
F		
G		

A. A ferramenta ideal para fornecer o serviço desejado é um Stored Procedure (SP).
VERDADEIRO

Una SP permite agrupar múltiples operaciones DML con control de transacción, errores y lógica condicional, que es exactamente lo que los requisitos i–iv exigen.

B. Sem o requisito iii e deixando os funcionários sem supervisor (no requisito ii), conseguíamos implementar o processo com um Trigger do tipo After Delete na tabela Employee.
VERDADEIRO

Sin el requisito iii (que cancela la operación), un Trigger AFTER DELETE podría ejecutar los DELETEs en Dependent y los UPDATEs de supervisor automáticamente al borrar el empleado.

C. A ferramenta a utilizar deve implementar as ações pela seguinte ordem: 1. verificar se o funcionário é gestor de departamento, abortando o processo em caso afirmativo; 2. delete do funcionário de Employee; 3. remover a referência nos

funcionários supervisionados; 4. delete de Dependent; 5. substituição do funcionário nos projetos associados pelo gestor do departamento.

FALSO

El orden es incorrecto. Primero deben eliminarse los registros dependientes (Dependent, WORKS_ON, referencias de supervisor) antes de borrar el empleado de Employee, para no violar las FK.

D. O requisito i podia ser implementado de forma declarativa no momento da criação da tabela Dependent, nomeadamente na definição da sua FK (Essn) para Employee.
VERDADEIRO

Definiendo ON DELETE CASCADE en la FK de Dependent(Essn) → Employee(Ssn), los dependientes se borran automáticamente al borrar el empleado.

E. Se implementarmos o processo com um Trigger Instead Of (no delete) em Employee, então o seguinte bloco T-SQL pode ser utilizado para verificar o critério iii:

```
sql

DECLARE @issn as char(9);

SELECT @issn = ssn FROM inserted;

if (select count(*) from Department where Mgr_ssn=@issn > 0)

begin

  -- error code here

end
```

FALSO

En un Trigger INSTEAD OF DELETE, la tabla lógica disponible es deleted, no inserted. La línea SELECT @issn = ssn FROM inserted es incorrecta.

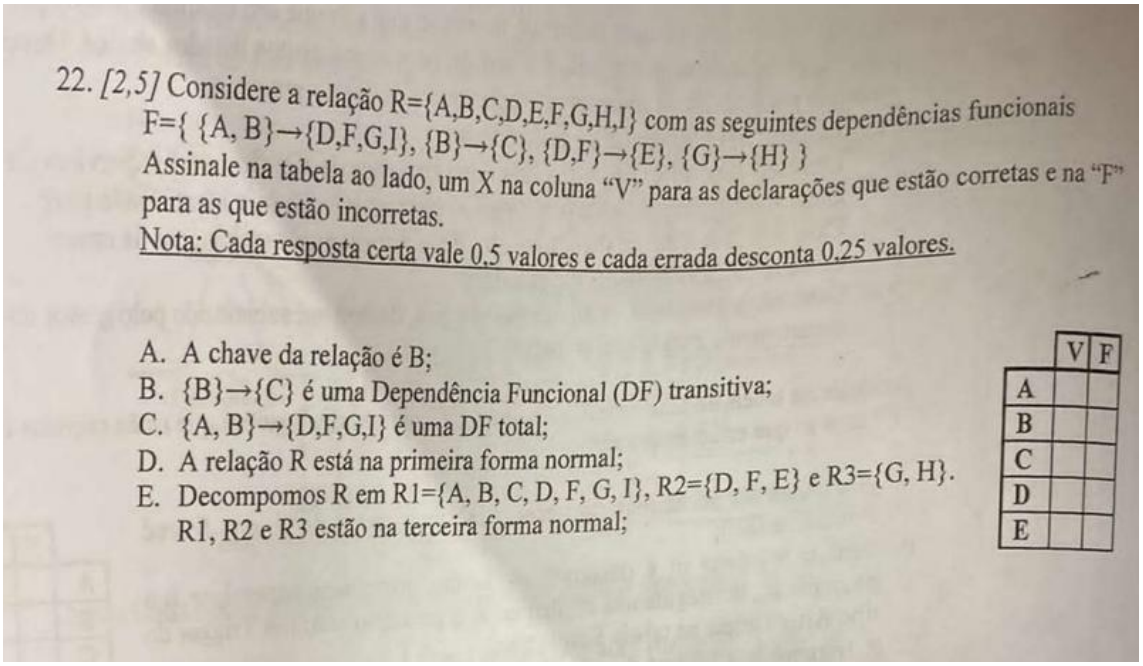
F. Não é possível implementar o processo com uma UDF do tipo escalar retornar um inteiro (0 – sucesso; 1 – erro).
VERDADEIRO

Las UDFs escalares en SQL Server no pueden contener operaciones DML (INSERT, UPDATE, DELETE) ni gestionar transacciones, por lo que no sirven para este proceso.

G. Numa implementação com SP e, caso a primeira instrução seja verificar se o funcionário é gestor de departamento (abortando o processo em caso afirmativo), então não temos necessidade de utilizar uma transação para garantir a atomicidade de todas as ações do processo.

VERDADEIRO

Si lo primero que hace la SP es verificar el requisito iii y abortar antes de modificar nada, entonces en el momento en que se ejecutan los DML no puede ocurrir ese error; aun así, se recomienda transacción para cubrir otros fallos posibles.



22. *[2,5]* Considere a relação R={A,B,C,D,E,F,G,H,I} com as seguintes dependências funcionais F={ {A, B}→{D,F,G,I}, {B}→{C}, {D,F}→{E}, {G}→{H} }
Assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas.
Nota: Cada resposta certa vale 0,5 valores e cada errada desconta 0,25 valores.

A. A chave da relação é B;
B. {B}→{C} é uma Dependência Funcional (DF) transitiva;
C. {A, B}→{D,F,G,I} é uma DF total;
D. A relação R está na primeira forma normal;
E. Decompomos R em R1={A, B, C, D, F, G, I}, R2={D, F, E} e R3={G, H}. R1, R2 e R3 estão na terceira forma normal;

	V	F
A		
B		
C		
D		
E		

A. FALSO — La clave es {A,B}, no B.

B. FALSO — {B}→{C} es una DF directa, no transitiva.

C. VERDADEIRO — Ningún subconjunto de {A,B} puede determinar {D,F,G,I} por sí solo.

D. VERDADEIRO — Por convención en estos ejercicios, se asume 1FN si no se indica lo contrario.

E. FALSO R1 no está en 3FN por la dependencia parcial {B}→{C}.

Teste 2019

1. Em SQL Server, podemos ter distintos DB *users* (um por cada DB) mapeados para o mesmo login.

VERDADEIRO: Un login puede tener un user diferente en cada base de datos.

2. Uma vista com 'WITH CHECK OPTION' garante que as condições da cláusula WHERE são verificadas numa operação de escrita (insert/update).

VERDADEIRO: Exactamente su propósito: evitar insertar/actualizar filas que queden fuera del filtro de la vista.

3. No controle de concorrência, o mecanismo de *locking* é um método do tipo otimista.

FALSO: El *locking* es un método pesimista (bloquea recursos antes de usarlos). El optimista no bloquea, valida al final.

4. Um *Trigger After* não deve ser utilizado quando a operação tem elevada probabilidade de ser *"rolled back"*.

VERDADEIRO: Si la operación se revierte frecuentemente, el trigger se ejecuta y luego se deshace, generando overhead innecesario. Mejor usar INSTEAD OF.

5. Num escalonamento concorrente, podemos ter situações de conflito quando temos transações simultâneas de consulta de dados.

FALSO: Dos operaciones de lectura read nunca entran en conflicto entre sí. Solo hay conflicto si al menos una operación es escritura.

6. Em SQL Server, um *Stored Procedure* pode ser utilizado como fonte de dados de uma consulta (i.e. na cláusula FROM de um SELECT).
FALSO: Una SP no puede usarse en el FROM de un SELECT. Para eso se usan funciones de tabla (*table-valued functions*).

7. Em SQL Server, o processo de *page split* ocorre quando se efetuam consultas (*queries*).

FALSO: El *page split* ocurre en operaciones de inserción/actualización, no en consultas de lectura.

8. No modelo relacional, devemos evitar relacionamentos entre duas relações que não sejam baseados em atributos chave primária e estrangeira.

VERDADEIRO: Los JOINs deben basarse en FK/PK para mantener integridad referencial. JOINs por atributos no clave son válidos, pero no recomendables como diseño.

9. Em termos de SQL DCL, podemos cancelar (anular) um DENY ou um GRANT com um REVOKE.

VERDADEIRO: REVOKE elimina tanto un GRANT como un DENY previamente concedido.

10. Num *Trigger After Update*, a tabela lógica *deleted* não contém tuplos (i.e. está vazia).

FALSO: En un AFTER UPDATE, la tabla deleted contiene los valores anteriores (antes del update) y inserted los nuevos. Nunca está vacía.

11. Um *Checkpoint* é uma marca que representa o momento em que o SGBD escreve para o disco o *Transaction Log*.

FALSO: El checkpoint escribe al disco las páginas de datos modificadas (*dirty pages*) del buffer, no el Transaction Log (que se escribe continuamente).

12. Uma *UDF* do tipo 'Escalar' não pode ser utilizada numa restrição de integridade do tipo *Check*.

VERDADEIRO: En SQL Server, las UDFs escalares no están permitidas en restricciones CHECK directamente sobre columnas de tabla.

13. Um cursor permite percorrer sequencialmente os tuplos retornados por uma *query* definida para o efeito.

VERDADEIRO : Definición exacta de cursor.

14. Em SQL Server, os *Triggers* são disparados uma vez por cada tuplo afetado pela operação (insert/update/delete).

FALSO: En SQL Server los triggers se disparan una vez por operación (no por fila). Las tablas inserted y deleted pueden contener múltiples filas.

15. Uma leitura suja (*dirty-read*) ocorre quando T2 acede a um elemento modificado por T1, sendo que T1 acaba por ser *"rolled back"*.

VERDADEIRO : Definición exacta de *dirty read*.

16. Uma aplicação deve promover a utilização de SQL dinâmico contendo elementos fornecidos pelos utilizadores para agilizar a interação com a base de dados.

FALSO: Exactamente lo contrario: usar SQL dinámico con inputs del usuario favorece SQL Injection. Se deben usar parámetros o SPs.

17. Numa transação podemos ter um *Rollback* implícito quando ocorre um erro numa instrução SQL da transação.

VERDADEIRO: Ciertos errores graves (como violaciones de constraint) pueden generar un rollback implícito de la instrucción o de toda la transacción.

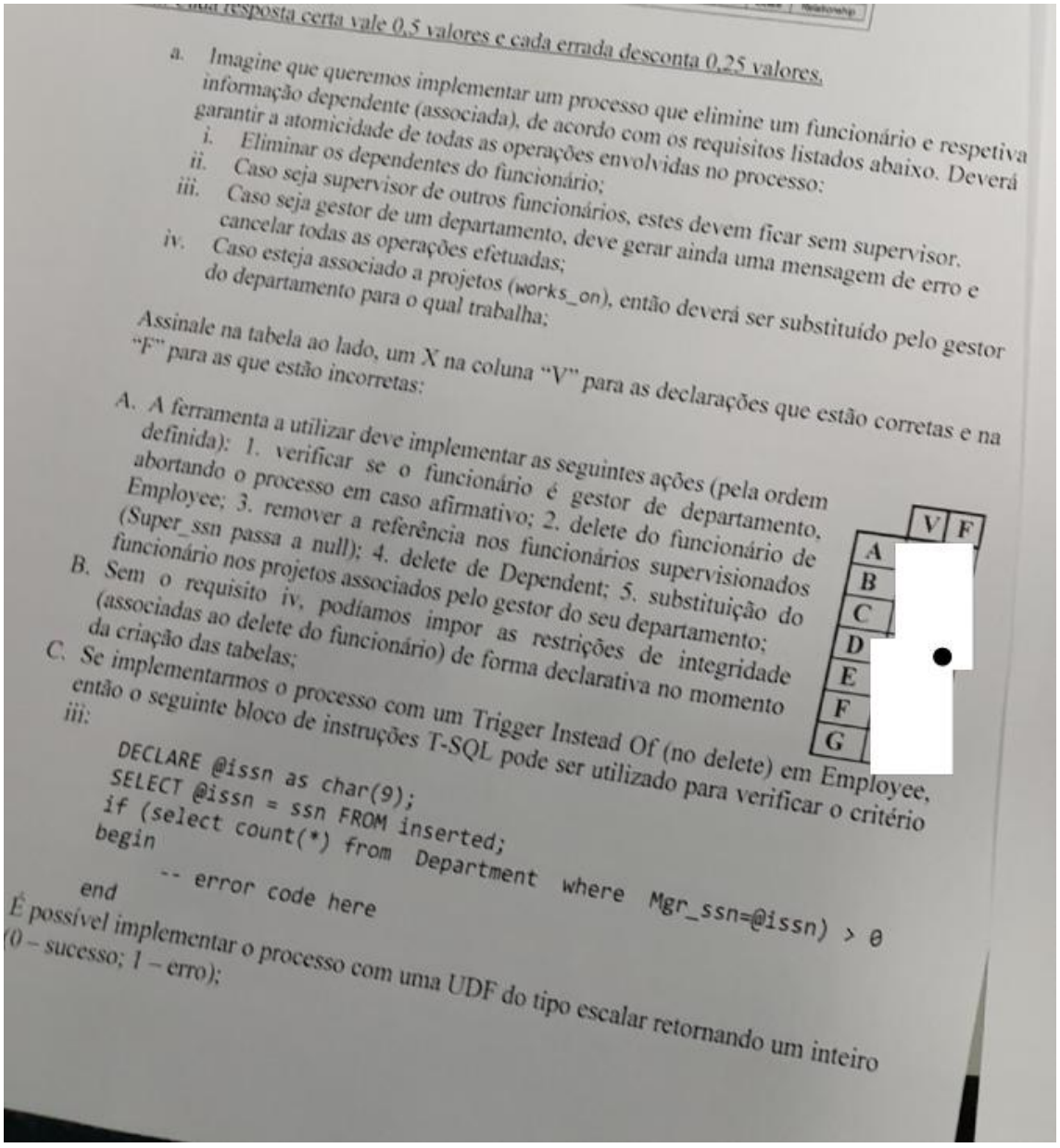
18. Em SQL Server, um atributo definido como *unique* não aceita valores *null* repetidos.

FALSO: En SQL Server, una restricción UNIQUE sí permite múltiples NULLs .

19. A variável global @@rowcount permite saber quantas linhas de código já foram executadas até ao momento na corrente *batch*.
FALSO: @@ROWCOUNT indica el número de filas afectadas por la última instrucción, no líneas de código ejecutadas.

20. Em SQL Server, não podemos definir o nível de isolamento de uma transação.

FALSO: Sí se puede con SET TRANSACTION ISOLATION LEVEL (READ UNCOMMITTED, REPEATABLE READ, SERIALIZABLE, etc.).



Cada resposta certa vale 0,5 valores e cada errada desconta 0,25 valores.

a. Imagine que queremos implementar um processo que elimine um funcionário e respetiva informação dependente (associada), de acordo com os requisitos listados abaixo. Deverá garantir a atomicidade de todas as operações envolvidas no processo:
   i. Eliminar os dependentes do funcionário;
   ii. Caso seja supervisor de outros funcionários, estes devem ficar sem supervisor.
   iii. Caso seja gestor de um departamento, deve gerar ainda uma mensagem de erro e cancelar todas as operações efetuadas;
   iv. Caso esteja associado a projetos (`works_on`), então deverá ser substituído pelo gestor do departamento para o qual trabalha;

Assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas:

	V	F
A		
B		
C		
D		
E		
F		
G		

A. A ferramenta a utilizar deve implementar as seguintes ações (pela ordem definida): 1. verificar se o funcionário é gestor de departamento, abortando o processo em caso afirmativo; 2. delete do funcionário de Employee; 3. remover a referência nos funcionários supervisionados (Super_ssn passa a null); 4. delete de Dependent; 5. substituição do funcionário nos projetos associados pelo gestor do seu departamento;

B. Sem o requisito iv, podíamos impor as restrições de integridade (associadas ao delete do funcionário) de forma declarativa no momento da criação das tabelas;

C. Se implementarmos o processo com um Trigger Instead Of (no delete) em Employee, então o seguinte bloco de instruções T-SQL pode ser utilizado para verificar o critério iii:

```
DECLARE @issn as char(9);
SELECT @issn = ssn FROM inserted;
if (select count(*) from Department where Mgr_ssn=@issn) > 0
begin
    -- error code here
end
```

É possível implementar o processo com uma UDF do tipo escalar retornando um inteiro (0 – sucesso; 1 – erro);

E. Numa implementação com SP e, caso a primeira instrução seja verificar se o funcionário é gestor de departamento (abortando o processo em caso afirmativo), então não temos necessidade de utilizar uma transação para garantir a atomicidade de todas as ações do processo;
F. A ferramenta ideal para fornecer o serviço desejado é um Stored Procedure (SP);
G. Não faz sentido implementar o processo com um Trigger do tipo After delete na tabela Employee;

b. Considere que todas as tabelas foram criadas (definidas), por defeito, com uma *primary key* e a base de dados tem habitualmente as seguintes consultas:
- Pesquisar os funcionários pelo Ssn ou pelo conjunto primeiro (Fname) e último nome (Lname).
- Pesquisar departamentos pelo seu número (Dnumber) ou pelo nome (Dname).
- Saber o nome dos projetos e número de horas para os quais determinado funcionário trabalha (Ssn).
- Obter o salário médio dos funcionários por género (Sex) e localização do departamento (Dlocation);

Tendo como referência o SQL Server e a criação de índices para as tabelas Employee (PK - Ssn), Department (PK - Dnumber) e Works_on (PK – Essn,Pno), assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas:

A. Devemos criar um índice *nonclustred* para o atributo Pno de Works_on;
B. Devemos criar um *nonclustered* para o atributo Dno de Employee;
C. Devemos criar dois índices *nonclustered* para os atributos Fname e Lname (um para cada) de Employee;
D. Devemos criar um índice *clustered* composto para os atributos (Fname, Lname) de Employee;
E. Devemos criar um índice *nonclustred* para o atributo Essn de Works_on;
F. Por defeito, será criado um *clustered index* para cada tabela;
G. Devemos criar um índice filtrado para os atributos Fname e Lname de Employee;
H. Se utilizarmos *unique* na definição do atributo Dname de Department, já não necessitamos de criar explicitamente um índice *nonclustred* para este atributo;

	V	F
A		
B		
C		
D		
E		
F		
G		
H		

a)

Contexto de los requisitos (del enunciado visto antes):

- **i.** Eliminar los dependientes del funcionario
- **ii.** Si es supervisor → otros funcionarios quedan sin supervisor (Super_ssn = null)
- **iii.** Si es gestor de departamento → error y cancelar todo
- **iv.** Si trabaja en proyectos → sustituirlo por el gestor del departamento

A. A ferramenta a utilizar deve implementar as seguintes ações (pela ordem definida): 1. verificar se é gestor → 2. delete de Employee → 3. remover referência nos supervisionados (Super_ssn = null) → 4. delete de Dependent → 5. substituição nos projetos pelo gestor.

❌ **FALSO** — El orden es incorrecto. El DELETE FROM Employee está en el paso 2, **antes** de borrar Dependent y actualizar Works_on. Eso viola las FK (Dependent y Works_on referencian Employee). Los dependientes y asignaciones a proyectos deben eliminarse/actualizarse **antes** de borrar el empleado.

B. Sem o requisito iv, podíamos impor as restrições de integridade (associadas ao delete do funcionário) de forma declarativa no momento da criação das tabelas.

✅ **VERDADEIRO** — Sin el req. iv (que requiere lógica de sustitución por el gestor), los requisitos i y ii se podrían implementar con ON DELETE CASCADE (para Dependent) y ON DELETE SET NULL (para Super_ssn en Employee). Son restricciones declarativas estándar de FK.

C. Se implementarmos o processo com um Trigger Instead Of (no delete) em Employee, então o seguinte bloco T-SQL pode ser utilizado para verificar o critério iii:

sql

```
DECLARE @issn as char(9);

SELECT @issn = ssn FROM inserted;

if (select count(*) from Department where Mgr_ssn=@issn) > 0

begin

  -- error code here

end
```

❌ **FALSO** — En un Trigger INSTEAD OF DELETE, la tabla lógica que contiene las filas que se intentan borrar es **deleted**, no inserted. La línea SELECT @issn = ssn FROM inserted es incorrecta; debería ser FROM deleted.

D. É possível implementar o processo com uma UDF do tipo escalar retornando um inteiro (0 – sucesso; 1 – erro).

❌ **FALSO** — Las UDFs escalares en SQL Server **no permiten** operaciones DML (INSERT, UPDATE, DELETE) ni gestión de transacciones. Solo pueden hacer consultas de lectura, por lo que son incapaces de ejecutar este proceso.

E. Numa implementação com SP e, caso a primeira instrução seja verificar se o funcionário é gestor de departamento (abortando o processo em caso afirmativo), então não temos necessidade de utilizar uma transação para garantir a atomicidade de todas as ações do processo.

✅ **VERDADEIRO** — Si lo **primero** que hace la SP es verificar el req. iii y abortar **antes de ejecutar cualquier DML**, entonces cuando llega al bloque de modificaciones ya es imposible que ocurra ese error. Sin embargo, la transacción sigue siendo recomendable para cubrir otros fallos inesperados en los pasos siguientes.

F. A ferramenta ideal para fornecer o serviço desejado é um Stored Procedure (SP).
✅ **VERDADEIRO** — Una SP permite combinar múltiples operaciones DML, lógica condicional (IF), control de errores (TRY...CATCH) y gestión de transacciones (BEGIN/COMMIT/ROLLBACK) en un único bloque atómico. Es exactamente la herramienta adecuada para este proceso.

G. Não faz sentido implementar o processo com um Trigger do tipo After Delete na tabela Employee.
✅ **VERDADEIRO** — El req. iii exige **cancelar** la operación si el empleado es gestor. Un Trigger AFTER se ejecuta **después** de que el DELETE ya ocurrió, por lo que para cancelarlo habría que hacer un ROLLBACK. Además, los Triggers AFTER no pueden impedir fácilmente la operación original. Lo correcto sería un INSTEAD OF o directamente una SP.

B)

Escenario: Tablas con PK por defecto — Employee (PK: Ssn), Department (PK: Dnumber), Works_on (PK: Essn, Pno). Consultas habituales: buscar por Ssn o (Fname, Lname); buscar departamentos por Dnumber o Dname; proyectos/horas por Ssn; salario medio por Sex y Dlocation.

A. Devemos criar um índice *nonclustered* para o atributo Pno de Works_on.
✅ **VERDADEIRO** — La PK es (Essn, Pno). El clustered index solo cubre búsquedas que empiezan por Essn. Para buscar solo por Pno se necesita un índice propio, ya que Pno no es el primer atributo de la clave.

B. Devemos criar um *nonclustered* para o atributo Dno de Employee.
✅ **VERDADEIRO** — La consulta de salario medio por Sex y Dlocation hace JOIN entre Employee y Department a través de Dno. Sin índice en Dno, ese JOIN obliga a un full scan de Employee.

C. Devemos criar dois índices *nonclustered* para os atributos Fname e Lname (um para cada) de Employee.
❌ **FALSO** — Las búsquedas son siempre por (Fname, Lname) **juntos**. Lo correcto es un

único índice compuesto (Fname, Lname). Dos índices separados no cubren bien búsquedas combinadas y añaden overhead en escrituras.

D. Devemos criar um índice *clustered* composto para os atributos (Fname, Lname) de Employee.
❌ **FALSO** — Employee ya tiene su clustered index en la PK (Ssn). En SQL Server solo puede existir **un clustered index por tabla**, por lo que crear otro es imposible.

E. Devemos criar um índice *nonclustered* para o atributo Essn de Works_on.
❌ **FALSO** — Essn es el **primer atributo** de la PK compuesta (Essn, Pno). El clustered index ya está ordenado por Essn primero, por lo que búsquedas por Essn ya están perfectamente cubiertas sin índice adicional.

F. Por defeito, será criado um *clustered index* para cada tabela.
✅ **VERDADEIRO** — En SQL Server, definir una PRIMARY KEY crea automáticamente un *unique clustered index* por defecto.

G. Devemos criar um índice filtrado para os atributos Fname e Lname de Employee.
❌ **FALSO** — Un índice filtrado indexa solo un subconjunto de filas usando WHERE. Aquí se necesita indexar **todos** los empleados por (Fname, Lname), por lo que lo correcto es un índice nonclustered compuesto normal, sin filtro.

H. Se utilizarmos *unique* na definição do atributo Dname de Department, já não necessitamos de criar explicitamente um índice *nonclustered* para este atributo.
✅ **VERDADEIRO** — En SQL Server, la restricción UNIQUE crea automáticamente un índice nonclustered sobre el atributo. Las búsquedas por Dname ya quedan cubiertas sin crearlo manualmente.